W

← Back

Q1 Destructor

10 marks

Determine the exact destruction order. class Employee { public: Employee() {} ~Employee() { cout << "Employee "; } }; class Manager { Employee e1; Employee e2; public: ~Manager() { cout << "Manager "; } };

Your Answer

Manager Employee Employee

RVIEW

Q2 **Operator** 10 marks

Which increment operation does this represent, and what would you change for postfix counter++?

Your Answer

Prefix increments first and returns the updated value; postfix returns the original value and then increments.

Q3 **Execute** 10 marks

Which destructors execute? class A { public: ~A() { cout << "A "; } }; class B { public: ~B() { cout << "B "; } }; int main() { try { A a; B b; throw 10; } catch (...) { cout << "Exception"; } }

Your Answer

Answer: BAException

Destructors execute in reverse order of object creation:

B::~B() → prints B

N

Q4 Operator

Which operation is prevented by = delete in class A { public: A() {} A(const A&) = delete; }; int main() { A a; A b = a; }

Your Answer

The operation prevented is assignment (=).

Because the class has a deleted copy-assignment operator:

A& operator=(const A&) = delete;

Q5 **Find**

Find the design flaw. class Employee { public: void calculateSalary(); void saveToDatabase(); void generateReport(); void sendEmail(); };

Your Answer

This class violates the Single Responsibility Principle (SRP).

Employee is handling too many responsibilities:

calculateSalary() → salary calculation

Q6 Compile

Can this code compile? Why? class A { public: virtual void show() = 0; virtual ~A() = 0; }; A::~A() {} class B : public A { public: void show() override {} };

Your Answer

Yes, it compiles.

A is an abstract class because show() and ~A() are pure virtual.
B overrides show() and provides its own destructor, so B is concrete and can be instantiated.

Answer: Yes, the code compiles.

Q7 **Output**

What is the output? class A { public: A() { cout << "A "; } A(const A&) { cout << "C "; } A& operator=(const A&) { cout << "E "; return *this; } }; int main() { A a; A b = a; A c; c = a; }

Your Answer

Output: ACC

A a; → constructor prints A

A b = a; → copy constructor prints C

Q8 **Output**

10 ma

Predict the output. class Number { int value; public: Number(int v) : value(v) {} Number operator+(const Number& n) { return Number(value + n.value); } void print() { cout << value; } }; int main() { Number a(10), b(20); Number c = a + b; c.print(); }

Your Answer

Output: 30

a + b calls the overloaded operator+():

10 + 20 = 30

Q9 Output

What is the output? class Base { public: Base() { show(); } virtual void show() { cout << "Base"; } }; class Derived : public Base { public: void show() override { cout << "Derived"; } }; int main() { Derived d; }

Your Answer

The object d is of class Derived, so its overridden show() function is called.

Answer: Derived

Q10 Constructor

Which constructor is called? class Test { public: Test() { cout << "Default"; } Test(int x) { cout << "Parameterized"; } Test(const Test&) { cout << "Copy"; } }; int main() { Test a; Test b(10); Test c = b; }

Your Answer

Test a; → calls Default constructor

Test b(10); → calls Parameterized constructor

Test c = b; → calls Copy constructor